

Day 11 — Coverage Models

FSM coverage for the sequence detector, the FIFO count and cross-coverage plans, the feature table, and branch vs. functional coverage.

Item 1 — Sequence Detector: States, Transitions, and Coverage Model

States (5): IDLE, S1, S10, S101, S1011

Transitions (10 total — 5 states × 2 input values):

IDLE→IDLE(0), IDLE→S1(1), S1→S10(0), S1→S1(1), S10→IDLE(0), S10→S101(1), S101→S10(0), S101→S1011(1, match), S1011→S10(0), S1011→S1(1, overlap).

Coverage model

- **Coverpoint on state** — bins for each of the 5 states. Says: “the FSM actually visited every state at least once,” not just the ones on the obvious happy path. In particular the DETECT state S1011 is only reached when a full match completes.
- **Coverpoint on the transition arc (state, din) → next_state** — 10 bins, one per row of the transition table. This is the *strongest* FSM metric (Learning Topic 11.2) — visiting every state doesn’t guarantee every *arc into and out of* that state was exercised, and a bug could hide specifically in one untested arc (e.g., the self-loops IDLE→IDLE and S1→S1 are easy to under-test if stimulus is biased toward “always making progress,” and the overlap arc S1011→S1 only fires when a 1 immediately follows a completed match).
- **Coverpoint on detected** — bins for 0 and 1. Sounds trivial, but it’s the difference between “the FSM reached S101” and “the FSM actually entered S1011 and fired the output” — these are not the same claim, and only the second one is what the design exists to do.
- **Coverpoint on “cycles between two consecutive detected pulses”** — a bin specifically for a 3-cycle gap (the minimum possible, since the overlap is 1 bit out of a 4-bit pattern). This is deliberately narrower than “did detected fire twice” — it specifically confirms the *overlapping-match* corner case (Day 7’s corner case #1) was exercised, not just two isolated matches with lots of idle time in between. The seq_detect_1011_overlap_tb hits exactly this (pulses in cycles 5 and 8).

“Fully exercised” for this design means all 5 state bins, all 10 transition bins, both detected bins, and the 3-cycle-gap overlap bin have all been hit at least once.

Item 2 — FIFO: Coverage Plan for count

Following Learning Topic 11.6’s datapath guidance (boundary values 0, 1, max-1, max, plus mid-range) rather than one big “everything in the middle” bin:

Bin	Range	Why it matters
empty	{0}	The most important boundary — confirms the FIFO actually reaches fully drained, not just “low.”
almost_empty	{1}	One entry — boundary-adjacent. Off-by-one bugs love to hide right next to the true boundary rather than at it.
mid	{[2:6]}	Comfortable middle ground — general functional confidence, not a specific edge.
almost_full	{7} (DEPTH-1)	Boundary-adjacent on the other side — same off-by-one reasoning as almost_empty.
full	{8} (DEPTH)	The other critical boundary — confirms the FIFO actually reaches completely full, not just “high.”

Five bins instead of the minimal three (**empty**/**mid**/**full**) specifically because the two adjacent-to-boundary bins (**1** and **DEPTH-1**) are exactly where an off-by-one bug (like **ISSUE-D5-02** on the Traffic Light) would be most likely to hide undetected if only the exact boundary values were checked.

Item 3 — Cross Coverage: **wr_en** × **rd_en** (FIFO)

cross wr_en, rd_en — 4 bins:

Bin	wr_en	rd_en	Why it matters
both_idle	0	0	Confirms the FIFO correctly holds steady when nothing is requested — the “do nothing” default branch must actually be exercised, not just assumed safe.
write_only	1	0	Core write path in isolation.
read_only	0	1	Core read path in isolation.
both	1	1	The trickiest interaction (CORNER_003) — confirms simultaneous access was actually driven, not just handled in code that nothing ever exercises.

Item 4 — FIFO Feature Table

Feature	Req-ID	Checker	Stimulus	Priority	Coverage Item
reset_initializes_fifo_to_empty	RST_001	Checker w/ expected value	Directed	P0	Coverpoint on (count, full, empty) immediately after reset deasserts; bin reset_state = {count==0, full==0, empty==1}.
write_increments_count_and_stores_data_correctly	FUNC_001	Scoreboard	Constrained-random	P0	Coverpoint on count during write-only cycles; bin write_progression = every value 1 through DEPTH reached via a write.
read_returns_data_in_fifo_order	FUNC_002	Scoreboard	Constrained-random	P0	Coverpoint on count <i>at the moment each read occurs</i> ; bins spanning 1 through DEPTH, confirming reads were exercised across the full range of fill levels, not just near-empty.
full_flag_asserts_when_count_equals_depth	CORNER_001	Checker w/ expected value	Directed	P0	Coverpoint on count, bin full = {DEPTH} (same bin as the count coverage plan above).
write_blocked_when_full	NEG_001	Assertion	Directed	P0	Cross coverage of wr_en × full; bin blocked_write = {wr_en=1, full=1}.
empty_flag_asserts_when_count_equals_zero	CORNER_002	Checker w/ expected value	Directed	P0	Coverpoint on count, bin empty = {0} (reached <i>after</i> having been non-zero, not just at power-on).
read_blocked_when_empty	NEG_002	Assertion	Directed	P0	Cross coverage of rd_en × empty; bin blocked_read = {rd_en=1, empty=1}.
count_unchanged_on_simultaneous_read_write	CORNER_003	Assertion	Directed	P1	Cross coverage of wr_en × rd_en × {full, empty, mid} status; bins simul_normal, simul_while_full, simul_while_empty.

Item 5 — 100% Branch Coverage vs. 100% Functional Coverage

100% branch coverage means every `if/else` and `case` branch in the RTL source has been taken at least once — a structural, code-level metric the simulator computes automatically just by watching which lines executed, with zero knowledge of what the design is actually supposed to do. 100% functional coverage means every scenario the DV engineer explicitly defined as meaningful (coverpoints, bins, crosses) has been exercised — a scenario-level metric that requires a human to have deliberately modeled what “meaningful” means, since the simulator can’t infer intent on its own. You can absolutely have one without the other, in both directions. You can have 100% branch coverage with far less than 100% functional coverage: every branch of the FIFO’s case statement could be hit while only ever testing `count` values near the middle of the range, never once at `count==0` or `count==DEPTH` — every line ran, but two of the design’s most important corner cases were never functionally verified. Conversely, you can have strong functional coverage with less than 100% branch coverage if there’s dead or redundant code nobody wrote a coverpoint for — functional coverage only measures scenarios someone thought to define, so it can’t rescue code paths nobody modeled. Branch coverage tells you the code *ran*; functional coverage tells you the design’s *meaning* was tested. Neither implies the other, which is exactly why real verification plans track both.